

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

DBMS PROJECT REPORT

1. INTRODUCTION

The Product and Category Management System is a database management system designed to organize and manage product information in an e-commerce environment.

In an e-commerce system, products need to be properly organized according to their categories. This project uses two main tables, Category and Product, to store and manage the information.

The system establishes a relationship between products and their respective categories. This makes it easier to add new products, update existing product information, delete unwanted products, and generate category-wise reports.

2. OBJECTIVE OF THE PROJECT

- To create a structured database for managing products and categories.
- To store category information efficiently.
- To store product details in the Product table.
- To establish a relationship between products and categories.
- To maintain data consistency using database constraints.
- To perform CRUD operations on product records.
- To generate useful category-wise product reports.
- To analyze product prices and product counts for each category.

3. PROJECT SCENARIO

The e-commerce database needs to organize product information according to different categories.

The system maintains the details of every product and establishes a proper connection between products and their respective categories.

For example, a category such as Electronics can contain multiple products such as laptops, headphones, mobile phones, and keyboards. Similarly, a Clothing category can contain shirts, jeans, dresses, and shoes.

This structure makes product management easier and allows category-based information to be retrieved using SQL queries.

4. DATABASE DESIGN

The database consists of two main tables:

- Category Table
- Product Table

These tables are connected using a foreign key relationship.

CATEGORY TABLE

The Category table stores information about the different categories available in the e-commerce system. Each category is assigned a unique CATEGORY_ID, which is used to identify the category.

The CATEGORY_ID acts as the Primary Key of the Category table.

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),DESCRIPTION VARCHAR(50));  
Table created.
```

PRODUCT TABLE

The Product table stores information about the products available in the e-commerce system. Each product has a unique PRODUCT_ID and is associated with a category using CATEGORY_ID.

The PRODUCT_ID acts as the Primary Key, while CATEGORY_ID acts as the Foreign Key.

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);  
Table created.
```

5. RELATIONSHIP BETWEEN TABLES

The relationship between the Category and Product tables is One-to-Many (1:N). This means that one category can have multiple products, while each product belongs to one category.

For example: Electronics → Mobile Phone, Laptop, Headphones.

The CATEGORY_ID in the Product table connects each product with the appropriate category in the Category table.

6. KEYS AND CONSTRAINTS

PRIMARY KEY

A Primary Key uniquely identifies each record in a table. In this project, CATEGORY_ID is the Primary Key of the Category table and PRODUCT_ID is the Primary Key of the Product table.

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

FOREIGN KEY

A Foreign Key establishes a connection between two related tables. In the Product table, CATEGORY_ID is used as the Foreign Key. It connects the Product table with the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT Fk_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

NOT NULL CONSTRAINT

The NOT NULL constraint ensures that important columns cannot be left empty. For example, product names and category names can be defined as NOT NULL because these values are required for proper identification.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK CONSTRAINT

The CHECK constraint restricts the values that can be entered into a column. For example, the product price can be restricted to positive values.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE CONSTRAINT

The UNIQUE constraint prevents duplicate values from being entered into a column. For example, category names can be maintained as unique values.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);  
Table altered.
```

7. INSERTING VALUES

After creating the Category and Product tables and applying the required constraints, records are inserted into the database.

The INSERT statement is used to add new category and product information. The Category table should be populated first, followed by the Product table, because the Product table contains a Foreign Key .

Category Table:

```
SQL> INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description');  
Enter value for category_id: 01  
Enter value for category_name: Home Appliances  
Enter value for description: Appliances used at home  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('01', 'Home Appliances ', ' Appliances used at home ')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: 02  
Enter value for category_name: Beauty  
Enter value for description: Beauty and Personal Care Products  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('02', 'Beauty', 'Beauty and Personal Care Products')  
  
1 row created.  
  
SQL> 03  
SP2-0226: Invalid line number  
SQL> /  
Enter value for category_id: 03  
Enter value for category_name: Grocery  
Enter value for description: Food and Daily grocery items  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('03', 'Grocery', 'Food and Daily grocery items')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: 04  
Enter value for category_name: Toys  
Enter value for description: Toys and games for children  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('04', 'Toys', 'Toys and games for children')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: 05  
Enter value for category_name: Footwear  
Enter value for description: Shoes, sandals and other footwear  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('05', 'Footwear', 'Shoes, sandals and other footwear')  
  
1 row created.
```

Product Table:

```
SQL> INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity);
Enter value for product_id: P01
Enter value for product_name: Mixer Grinder
Enter value for category_id: 01
Enter value for price: 3500
Enter value for stock_quantity: 15
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P01', 'Mixer Grinder', '01', 3500, 15)

1 row created.

SQL> /
Enter value for product_id: P02
Enter value for product_name: Electric Kettle
Enter value for category_id: 01
Enter value for price: 1800
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P02', 'Electric Kettle', '01', 1800, 20)

1 row created.

SQL> /
Enter value for product_id: P03
Enter value for product_name: Face Cream
Enter value for category_id: 02
Enter value for price: 450
Enter value for stock_quantity: 30
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P03', 'Face Cream', '02', 450, 30)

1 row created.

SQL> /
Enter value for product_id: P04
Enter value for product_name: Basmathi Rice
Enter value for category_id: 03
Enter value for price: 750
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P04', 'Basmathi Rice', '03', 750, 50)

1 row created.
```

```
SQL> /
Enter value for product_id: P05
Enter value for product_name: Remote car
Enter value for category_id: 04
Enter value for price: 1200
Enter value for stock_quantity: 25
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P05', 'Remote car', '04', 1200, 25)

1 row created.
```

```
SQL> SELECT Category_ID, Category_Name, Description
2 FROM Category;
```

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
01	Home Appliances	Appliances used at home
02	Beauty	Beauty and Personal Care Products
03	Grocery	Food and Daily grocery items
04	Toys	Toys and games for children
05	Footwear	Shoes, sandals and other footwear

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
P01	Mixer Grinder	01	3500	15
P02	Electric Kettle	01	1800	20
P03	Face Cream	02	450	30
P04	Basmathi Rice	03	750	50
P05	Remote car	04	1200	25

8. CRUD OPERATIONS

CRUD stands for Create, Read, Update, and Delete. These are the basic operations used to manage records in a database.

Insert

The Create operation is used to add a new product to the Product table.

```
SQL> INSERT INTO Product (PRODUCT_ID, PRODUCT_NAME, CATEGORY_ID, PRICE, STOCK_QUANTITY) VALUES ('P06', 'Wheat Flour', '03', '450', '45');
```

```
1 row created.
```

UPDATE

The Update operation is used to modify existing product information.

```
SQL> UPDATE Product SET Price = 600 WHERE Product_Name = 'Face Cream';
```

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

CATEGORY_NAME	PRODUCT_COUNT
Home Appliances	2
Beauty	1
Grocery	2

9. CATEGORY-WISE PRODUCT REPORTS

SQL queries can be used to generate useful reports based on product categories. These reports help in understanding the products available in each category and analyzing their prices.

DISPLAY ALL PRODUCTS UNDER EACH CATEGORY

This report displays products along with their corresponding categories.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity FROM product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Home Appliances	Mixer Grinder	3500	15
Home Appliances	Electric Kettle	1800	20
Beauty	Face Cream	600	30
Grocery	Basmathi Rice	750	50
Grocery	Wheat Flour	450	45

COUNT NUMBER OF PRODUCTS IN EACH CATEGORY

This query determines the number of products available in every category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name
```

CATEGORY_NAME	PRODUCT_COUNT
Home Appliances	2
Beauty	1
Grocery	2

FIND THE HIGHEST-PRICED PRODUCT IN EACH CATEGORY

This report identifies the product with the highest price within each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_price FROM product p JOIN Category c ON p.Category_ID = c.category_ID WHERE p.Price =(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Home Appliances	Mixer Grinder	3500
Beauty	Face Cream	600
Grocery	Basmathi Rice	750

DISPLAY CATEGORIES HAVING MORE THAN 5 PRODUCTS

This query identifies categories containing more than five products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 2;
```

no rows selected

FIND AVERAGE PRODUCT PRICE CATEGORY-WISE

This query calculates the average product price for each category.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Home Appliances	2650
Beauty	600
Grocery	600

10. ADVANTAGES OF THE SYSTEM

- Easy Product Management – Product information can be easily added, updated, viewed, and deleted.
- Category-Based Organization – Products are organized according to their respective categories.
- Data Consistency – Primary keys, foreign keys, and other constraints help maintain consistent data.
- Easy Data Retrieval – SQL queries can quickly retrieve required product and category information.
- Useful Reports – Category-wise reports provide information about product counts and prices.
- Reduced Data Duplication – Separate Category and Product tables help avoid unnecessary repetition of category information.

11. RESULT

The Product and Category Management System successfully organizes product information using a relational database.

The Category and Product tables were created and connected using a One-to-Many relationship. Primary Key, Foreign Key, NOT NULL, CHECK, and UNIQUE constraints were used to maintain data integrity.

CRUD operations were performed to manage product records. SQL queries were also used to generate category-wise reports such as product count, highest price, and average price.

12. CONCLUSION

The Product and Category Management System provides a structured method for managing e-commerce product information.

The relationship between the Category and Product tables ensures that products are properly associated with their categories. Database constraints help maintain data accuracy and consistency.

CRUD operations make it possible to manage product records throughout their lifecycle, while category-wise SQL queries provide useful information for analyzing products and their prices.

Therefore, the project demonstrates the practical use of DBMS concepts, relational tables, keys, constraints, relationships, SQL queries, and CRUD operations in an e-commerce environment.